

区块链侧链技术（0）——衍生知识补充

网络层

Layer	内容	层
layer 2：链下扩展	可编程	应用层
	堆栈脚本 算法 智能合约	合约层
layer 1：链上扩展	分配机制 发行机制	激励层
	PoA Raft PoW PoS	共识层
	P2P网络 传播机制 验证机制	网络层
	区块数据 链式结构 时间戳哈希函数 Merkle树 非对称加密	数据层
layer0：第0层	OSI模型中传输层与网络层	

区块链平台

区块链平台	Bitcoin	Ethereum	Hyperledger Fabric
许可限制	非许可链	由配置决定	许可链
数据限制	公有链	由配置决定	联盟链
共识算法	PoW	PoW、PoA、PoS	由配置决定
匿名性	匿名	匿名	非匿名
原生数字货币	比特币	以太（以太币）	无
可编程性	有限制，堆栈脚本	图灵完备，虚拟机	图灵完备，docker 容器

Hyperledger Fabric

Hyperledger Fabric 网络中包括

客户端节点、CA 节点、Peer 节点和 Orderer 节点

1. Peer 节点分为主节点、背书节点、记账节点和锚节点。
2. 客户端节点提供 cli 命令行工具和多语言 SDK，与其他节点进行交互，发起交易操作。
3. CA 节点负责颁发证书，由 fabricca-server 和 fabric-ca-client 组成。
4. Peer 主节点负责与 Orderer 节点通信，从 Orderer 节点获取区块，并在 Peer 节点间进行同步。
 1. Peer 背书节点模拟执行交易，将签名后的执行结果打包返回。
 2. Peer 记账节点负责验证交易的正确性并更新本地账本数据。
 3. 一个通道中的锚节点用于发现通道中其他组织的节点。
 4. Orderer 节点接收背书节点签名的交易，对未打包的交易排序，出块，最后将区块广播到记账节点。

底层存储

大多数区块链状态存储都使用 goleveldb 数据库引擎

底层数据结构LSM-Tree

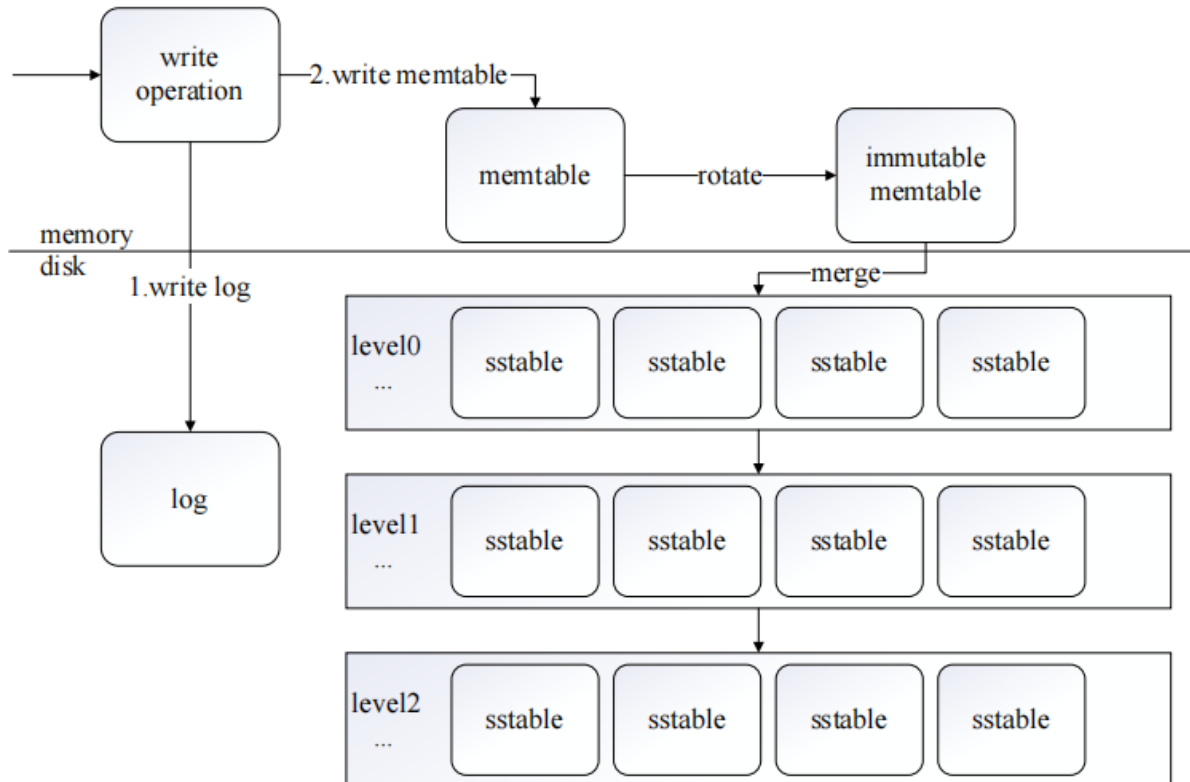
核心思想是将请求的写操作在内存中延迟并转换为批量写操作，最终将写入硬盘中的随机写操作转换为顺序写操作

goleveldb

1. 内存中有memtable 和 immutable memtable 两个跳表数据结构
2. 磁盘上有多个 level (L0-L6) ，每个 level 有多个 sstable 文件
3. 磁盘上还存有 WAL 日志文件 log、各 level 的元数据文件 MANIFEST、当前元数据文件的指针 current

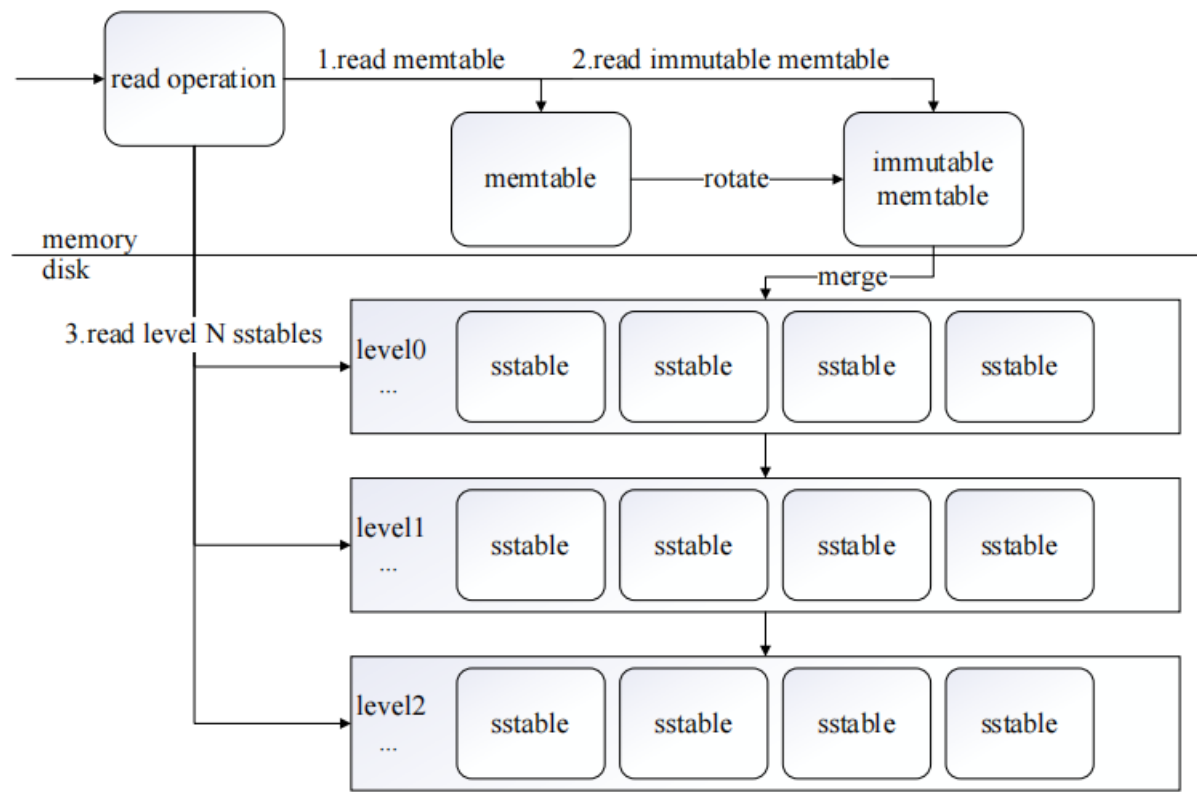
写入操作流程：

1. 首先 goleveldb 将键值数据存储在一个日志(log)文件中然后将其写入 memtable。
2. 当固定大小的 memtable 被写满时，goleveldb 切换到一个新的 memtable 和日志文件用以继续接收数据。
3. 接着在后台将写满的 memtable 转换为 immutable memtable，随后使用压缩线程将其落盘，在 level 0 上创建一个新的 sstable文件，最后丢弃日志文件。
4. 因为每个 goleveldb 需要清理无效的数据，并且保证键的有序性，所以设计了压缩操作对 level 中的 sstable 文件进行归并。



读取操作流程：

- 1. 首先在 memtable 中查找指定的 key
- 2. 其次在immutable memtable中查找指定的key
- 3. 最后按照由低到高的顺序在每个level中的sstable文件中查找指定的 key，其中使用布隆过滤器与索引信息加快 key 的查找速度。



LSM-Tree 写放大问题：

在 goleveldb 中的每个 level 的最大容量是前一个 level 的 10 倍。

例如：

在最坏的情况下，前一个 level 中的一个 sstable 文件被压缩时，和下一个 level 中的十个 sstable 文件进行归并，写放大为 10。需要被压缩的 level 从 level 1 到 level 6，共六层，最坏的情况下需要压缩 5 次，写放大为 50。

LSM-Tree 读放大问题：

指实际从磁盘中执行查找所读取的数据大小与请求读取的数据大小之比。

例如：

查询一个 1KB 的键值对时，goleveldb 需要读取 16KB 的索引信息块，4KB 的布隆过滤器块和 4KB 的数据块，在最坏的情况下需要读取 level 0 的 8 个 sstable 文件以及从 level 1 到 level 6 六层中每层各一个 sstable 文件，共 14 个 sstable文件，读放大等于 24*14，为 336 倍。

多方签名（P2SH脚本）

用于n个签名者对消息m进行签名。其构造如下： $\pi \leftarrow \text{Setup}(\lambda)$ ：初始化算法接受安全参数 λ 作为输入，输出公共系统参数 π 。 $(sk, pk) \leftarrow \text{KeyGen}(\pi)$ ：密钥算法接受公共系统参数 π 作为输入，输出用户的公私钥对 (sk, pk) 。

$\sigma \leftarrow \text{Sign}(\pi, (sk, pk), L, m)$: 签名算法被 n 个签名者执行, 该算法接受公共系统参数 π , 签名者的公私钥 (sk, pk) , n 个签名者的公钥集 $L = \{pk_1, \dots, pk_n\}$ 与待签名消息 m 作为输入, 输出多方签名 σ 。 $\{0, 1\} \leftarrow \text{Ver}(L, m, \sigma)$: 根据公共系统参数 π , 验证算法接受 n 个签名者的公钥集 L , 消息 m , 多方签名 σ 作为输入, 如果多方签名 σ 对 n 个签名者的公钥集 L 和消息 m 有效, 则输出 1, 若签名无效, 则输出 0。

跨链交互

实质上是将链 A 的数据安全可信地传输到链 B 的操作, 一般是资产 (比如余额) 数据

跨链一般有下面几种方式:

1. **原子交换 (Atomic Swaps)**: 原子交换是一种基于智能合约的技术, 允许不同区块链上的用户直接交换加密货币, 而无需信任第三方。参与者在各自的区块链网络上锁定资产, 然后在两个链上同时实现资产的原子交换。
2. **中间体/网关模型**: 通过引入中间体或网关机构, 不同区块链网络之间可以实现资产的转移和交换。这种模型通常需要用户信任中介方, 但能够提供更高的灵活性和互操作性。
3. **侧链 (Sidechains)**: 侧链是连接到主区块链的并行区块链, 用户可以在不同的侧链和主链之间转移资产。侧链技术可以实现更高的吞吐量和隐私性, 促进跨链交互。
4. **跨链协议 (Cross-Chain Protocols)**: 跨链协议旨在建立各种区块链之间的桥梁, 使它们可以相互通信和交换价值。例如, Polkadot、Cosmos 等项目提供了跨链通信和资产转移的协议。
5. **中继链 (Relay Chain)**: 中继链是连接多个不同区块链的链, 可以跨区块链进行资产转移和通信。例如, Polkadot 的中继链就可以实现不同平行链之间的跨链交换。

双向锚定技术

侧链作为一种跨链方式, 使用一种称为双向锚定的技术来与主链进行通信。

在双向锚定中, 数据的管理与释放有四种模式。分别是:

1. 单一托管模式

单一托管模式, 由主链确定**可信托管方** (如交易所等), 托管方接收到主链锁定信息或资产等事件时, 托管方将事件同步到侧链, 在侧链上解锁相应等价的信息或侧链Token 资产。

单一托管模式效率高, 流程简单, 但单一托管模式过于中心化, 因为所有的跨链操作都经过托管方, 如果托管方不诚实, 则会有风险发生的可能。

2. 联盟托管模式

联盟托管模式, 是对单一托管模式的改进。当托管方接收到主链信息或资产等事件时, 托管方中每个代表自行核实信息和资产情况, 只有在所有代表多方签名后, 才能将信息或资产进行转移。

联盟托管人解决了部分中心化的问题, 但仍需要从一开始就严格筛选可信的代表。

3. SPV模式

SPV 模式, 也称为简化支付验证, 源自比特币 **UTXO** 模型, 该模型通过少量数据 (MerkleProof) 来验证特定区块中是否存在某笔交易。在 SPV 模式, 用户将信息或资产发送到主链中的一个特殊地址, 在发送成功后, 侧链会接收到一个 SPV 证明, 以确保资产被锁定, 根据SPV 证明释放侧链中的资产。

SPV 模式去中心化, 但仅适用于 UTXO 模型作为状态转化方式的区块链框架。例如, 基于 UTXO 模型的比特币框架中使用 SPV 来证明交易的存在, 但在基于账户模型的以太坊中就不能直接使用 SPV。

4. 驱动链模式

在驱动链模式中，矿工作为资产托管方，由矿工监测区块链状态，矿工们共同决定资产的解锁与发送。

缺点是需要对已部署的区块链进行硬分叉。

之后的文章中，会介绍常用的跨链方法